



A utility function for propagating the most significant bit

Document number:

[P3764R0](#)

Date:

2025-07-15

Audience:

SG6

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3764/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/msb-to-mask.cow



ABSTRACT: Add a function to the `<bit>` header which converts the most significant bit to a bit mask.

§ Contents

- 1 **Introduction**
 - 1.1 This mask creation needs a function
- 2 **Motivation**
- 3 **Design**
 - 3.1 Support for signed and unsigned types
 - 3.2 Naming
 - 3.3 SIMD support
- 4 **Implementation**
- 5 **Wording**
 - 5.1 `[bit]`
 - 5.2 `[simd]`



§ 1. Introduction

In bit manipulation, it is a common technique to propagate the uppermost bit to all the other bits, creating a "bit mask". This bit mask can then be used to make an otherwise branching operation branchless.



EXAMPLE: The following two functions for computing the positive remainder (assuming y is positive) are equivalent:

```
// Naive implementation
int mod_naive(int x, int y) {
    int rem = x % y;
    if (rem < 0) rem += y;
    return rem;
}

// Branchless implementation
int mod_branchless(int x, int y) {
    int rem = x % y;
    rem += y & (rem >> (INT_WIDTH - 1));
    return rem;
}
```

Modern optimizing compilers often find such optimizations already, at least in simple examples like the one above. In fact, Clang optimizes both functions the same for x86_64. However:

- In larger examples, this becomes much less reliable, and `&&` and `if` do get translated into conditional jumps.
- Compilers only perform this transformation if it heuristically better. Branchless programming is often applied in scenarios where code must be protected against timing-based side channel attacks, and so branchless variants may be preferred even at an extra cost.
- Even modern optimizing compilers miss optimizations like these sometimes.

Therefore, using this masking technique directly is not obsolete. You can find many examples of this technique being used at [\[BitTwiddlingHacks\]](#) by searching for `sizeof` on that page.

§ 1.1. This mask creation needs a function

Creating a bit mask from the sign bit by hand is especially tedious compared to many other techniques in bit manipulation, at least when written directly using an arithmetic right-shift. That is because it requires right-shifting by the operand width minus one. In generic code that is meant to operate on different integer types, this requires use of

`std::numeric_limits<T>::digits`, which is quite verbose and difficult to use because it returns different results for signed and unsigned types.



Another possible way to make such a mask (for signed types only) is to write `(x < 0 ? -1 : 0)`, but this reintroduces reliance on the compiler to eliminate branches. It also doesn't express directly what is emitted by the compiler, and in the context of bit manipulation, keeping code "close to hardware" is often desirable.

Regardless of the implementation details, C++ users sometimes create a utility function/macro that performs sign-masking. [\[GitHubCodeSearch\]](#) for `-is:fork language:C++ "sign_mask("` reveals that ~8000 files already use a function called `sign_mask`, although not all of these functions have the functionality proposed here.

It would be nice if this function was provided by the standard already.

§ 2. Motivation

A sign-masking function should be included in the `<bit>` header because

- it is a useful utility, similar to `has_single_bit` and `bit_ceil`,
- C++ users sometimes make this function/macro themselves already,
- there is some disagreement over how to write it, and the specifics may depend on hardware capabilities,
- the technique is sometimes used in SIMD as well, and the implementation using comparison to zero is "closer to hardware" in that context; however, it is still the same operation, and a common SIMD/non-SIMD spelling would be more expressive, and
- writing this function correctly, generically, is tedious when using `std::numeric_limits`.

§ 3. Design

§ 3.1. Support for signed and unsigned types

As shown in the example in [§1. Introduction](#), this function is sometimes used with signed types, not just with unsigned types. That is why it should accept both.

Such an interface would be somewhat inconsistent with the remaining functions in `<bit>`, but permitting the use of signed types within `<bit>` *in general* is not within the scope of this paper. To my knowledge, another SG6 proposal which proposes use of signed types in `<bit>` is already being developed.

§ 3.2. Naming

I propose the name `std::msb_to_mask` because it expresses its effects very clearly:



`std::msb_to_mask` converts the most significant bit (MSB) into a bit-mask.



The function should behave bitwise-equivalently for signed and unsigned types, so including `sign` within the name would cause confusion because unsigned types have no sign. While the name `sign_mask` has some precedent, it also frequently refers to a mask where the uppermost bit is set, i.e. a mask *of* the sign bit.

§ 3.3. SIMD support

Following [P2933R4], almost all functions (e.g. excluding `bit_cast`) in `<bit>` should also have `std::simd` overloads. There is no compelling reason why the proposed function should break that pattern.

§ 4. Implementation

```
namespace std {
    template<signed-or-unsigned-integer T>
    constexpr T msb_to_mask(T x) noexcept {
        using S = make_signed_t<T>;
        return static_cast<T>(static_cast<S>(x) >> numeric_limits<S>::digits);
    }

    namespace simd {
        template<simd-type V>
        constexpr V msb_to_mask(const V& v) noexcept {
            using S = rebind_t<make_signed_t<typename V::value_type>, V>;
            return static_cast<T>(static_cast<S>(v) >> numeric_limits<S>::digits);
        }
    }
}
```



NOTE: The SIMD implementation is naive. On x86_64 with SSE, an optimal implementation would be based on e.g.:

```
_mm_cmplt_epi32(v, _mm_setzero_si128())
```

That is because comparison instructions like `_mm_cmplt_epi32` already yield bit-masks as results.

§ 5. Wording

Bump feature-test macros in [version.syn] as follows:



```
#define __cpp_lib_bitops 201907L 20XXXXL // freestanding, also in <bit>
#define __cpp_lib_simd 202502L 20XXXXL // also in <simd>
```



§ 5.1. [bit]

In [bit.syn], change the synopsis as follows:



```
namespace std {
    [...]

    // [bit.count], counting
    template<class T>
        constexpr int countl_zero(T x) noexcept;
    template<class T>
        constexpr int countl_one(T x) noexcept;
    template<class T>
        constexpr int countr_zero(T x) noexcept;
    template<class T>
        constexpr int countr_one(T x) noexcept;
    template<class T>
        constexpr int popcount(T x) noexcept;

    // [bit.mask], masks
    template<class T>
        constexpr T msb_to_mask(T x) noexcept;

    [...]
}
```

In [bit], add a new subclause immediately following [bit.count]:



Masks

[bit.mask]

```
template<class T>
    constexpr T msb_to_mask(T x) noexcept;
```

Constraints: T is a signed or unsigned integer type ([basic.fundamental]).

Effects: Equivalent to:

```
using S = make_signed_t<T>;
return static_cast<T>(static_cast<S>(x) >> numeric_limits<S>::dig
```

§ 5.2. [simd]

In [simd.syn], change the synopsis as follows:



```
[...]
```



```
// [simd.bit], Bit manipulation
template<simd-type V> constexpr V byteswap(const V& v) noexcept;
template<simd-type V> constexpr V bit_ceil(const V& v) noexcept;
template<simd-type V> constexpr V bit_floor(const V& v) noexcept;
template<simd-type V> constexpr V msb_to_mask(const V& v) noexcept;

[...]

// [simd.bit], Bit manipulation
using simd::byteswap;
using simd::bit_ceil;
using simd::bit_floor;
using simd::msb_to_mask;
using simd::has_single_bit;
using simd::rotr;
using simd::rotr;
using simd::bit_width;
using simd::countl_zero;
using simd::countl_one;
using simd::countr_zero;
using simd::countr_one;
using simd::popcount;

[...]
```

In [simd.bit], following the declaration of bit_floor, insert a new declaration as follows:



```
template<simd-type V> constexpr V msb_to_mask(const V& v) noexcept;
```

Constraints: The type `V::value_type` is a signed or unsigned integer type ([basic.fundamental]).

Returns: A `basic_vec` object where the i^{th} element is initialized to the result of `std::msb_to_mask(v[i])` ([bit.mask]) for all i in the range $[0, V::size())$.

§ 6. References

[N5008] Thomas Köppe. Working Draft, Programming Languages — C++ (2025-03-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf>

[BitTwiddlingHacks] Sean Eron Anderson. Bit Twiddling Hacks
<https://graphics.stanford.edu/~seander/bithacks.html>

[GitHubCodeSearch] GitHub code search for C++ uses of a "sign_mask" function
https://github.com/search?q=is%3Afork+language%3AC%2B%2B++++%22sign_mask%28%22&type=code

[P2933R4] Daniel Towner, Ruslan Arutyunyan. Extend <bit> header function with overloads for `std::simd` (2025-02-13)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2933r4.html>